Задача 1: Магазин с продукти и количка (Intermediate OOP)

Описание:
Създай класове за `Product` и `ShoppingCart`.

Изисквания:

1. `Product` има: `name`, `price`, `quantity_in_stock`.
2. `ShoppingCart` има:
   - метод `add_product(product, quantity)` → добавя продукт, ако има наличност.
   - метод `remove_product(product_name, quantity)` → премахва продукти от количката.
   - метод `total()` → изчислява общата цена.
   - проверка: ако се опиташ да добавиш повече, отколкото има в наличност → хвърли грешка.
3. Поддържай история на добавените продукти (например списък с имена и количества).
4. По желание: `apply_discount(discount_percentage)`.

Цел: да упражниш **инкапсулация, методи и валидиране на данни**.

◆ Задача 2: Система за управление на ученици и оценки (Intermediate/Advanced)

Описание:
Създай класове `Student` и `GradeBook`.

Изисквания:

1. `Student` има:
   - `name`, `grades` (списък от числа)
   - метод `add_grade(grade)` → добавя оценка между 2 и 6 (или 0–100)
   - метод `average()` → връща средна оценка
2. `GradeBook` има:
   - списък със студенти
   - метод `add_student(student)`
   - метод `get_student(name)` → връща обекта студент
   - метод `top_student()` → връща студента с най-висока средна оценка
3. Валидации: добавяне на оценки извън границите → грешка

Цел: упражняваш **композиция, списъци с обекти и работа с методи**.

◆ Задача 3: Мини банка с транзакции и история (Advanced OOP)

Описание:
Разширение на твоя проект с банкови акаунти, но добавяш история и такси.

Изисквания:

1. Всеки акаунт (`Checking`, `Savings`, `Business`) поддържа списък `transactions`, всеки е речник с:
   - `type` (`deposit`/`withdraw`)
   - `amount`
   - `balance_after`
   - `description` (по желание)
2. Добави метод `transfer_to(other_account, amount)` → прехвърля пари между акаунти.
3. Такса за `BusinessBankAccount` при всяко теглене: 1% от сумата.
4. `SavingsBankAccount` добавя лихва по команда `add_interest()`.
5. `show_transactions()` → показва всички транзакции красиво.

Цел: упражняваш **наследяване, композиция, списъци от обекти, изключения и бизнес логика**.

◆ Задача 4: Система за управление на работници и проекти (Hard)

Описание:
Създай система с `Employee` и `Project`.

Изисквания:

1. `Employee` има:
   - `name`, `role`, `salary`
   - метод `assign_project(project)` → добавя проект към списък с текущи проекти
   - метод `work()` → връща „Employee X is working on Y projects“
2. `Project` има:
   - `name`, `budget`, `employees` (списък)
   - метод `add_employee(employee)` → добавя служител към проекта
   - метод `project_status()` → показва колко служители и текущ бюджет
3. Валидации:
   - Един служител не може да работи по повече от 3 проекта едновременно
   - Проект не може да надхвърля бюджет X (по избор)

Цел: упражняваш **обекти, списъци от обекти, взаимовръзки между класове, OOP логика и валидации**.

◆ Задача 5: Library Management System (Expert)

Описание:
Създай система за библиотека с книги, читатели и заемания.

Изисквания:

1. Класове:
   - `Book`: `title`, `author`, `isbn`, `available_copies`
   - `Reader`: `name`, `borrowed_books` (списък)
   - `Library`: списък с книги и читатели
2. Методи:
   - `borrow_book(reader_name, book_title)` → проверява наличност и добавя към `borrowed_books`
   - `return_book(reader_name, book_title)` → връща книгата и обновява наличност
   - `add_book(book)` → добавя нова книга
   - `show_books()` → списък с книги и налични копия
3. Валидации:
   - Не можеш да заемаш книга, ако няма налични копия
   - Един читател не може да заема повече от 5 книги едновременно

Цел: Много обекти и взаимоотношения, OOP композиция, валидиране и бизнес логика – почти реален mini проект, тип „интервю задача“.